

# DVWA Pentest Lab Report

Environment:  
Kali Linux (Attacker VM)  
Ubuntu Server + DVWA (Target VM)

Prepared by:  
Aisha Duze

Date:  
22 May 2026

## EXECUTIVE SUMMARY

A security assessment was performed against a vulnerable web application called Damned Vulnerable Web Application referred to as DVWA hosted on an Ubuntu virtual machine. Multiple web vulnerabilities were successfully identified and exploited, including command injection and local file inclusion. The findings demonstrate risks associated with improper input validation and insecure application design.

## SCOPE

Target IP address:

192.168.146.248

Applications Tested: DVWA

Testing Environment:

- VirtualBox isolated homelab
- Kali Linux attacker VM
- Ubuntu Server target VM

Testing Type:

- Authenticated web application penetration testing

## METHODOLOGY

1. Reconnaissance & Enumeration with Nmap
2. Web Enumeration with gobuster & burpsuite
3. Vulnerability Testing/Exploitation
4. Evidence Collection
5. Remediation Recommendations

## TOOLS

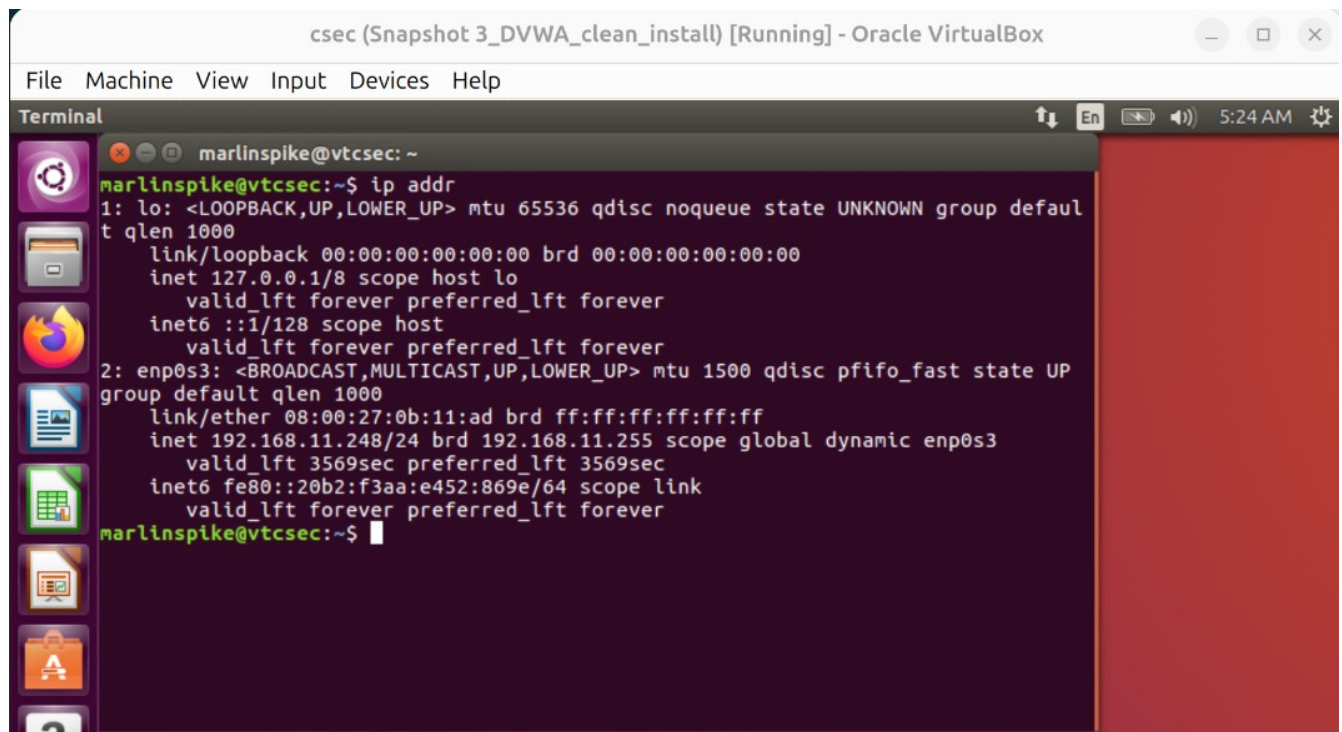
| Tool       | Purpose                       |
|------------|-------------------------------|
| Nmap       | Network/service enumeration   |
| Gobuster   | Directory discovery           |
| Burp Suite | HTTP interception             |
| Firefox    | Web testing                   |
| DVWA       | Vulnerable target application |

## FINDINGS

### PHASE 1- RECONNAISSANCE & ENUMERATION

1.

On the Ubuntu VM, I ran the 'ip addr' command which generated the IP address of the VM. I proceeded to find the target IP in the Kali VM and run the Nmap scans.

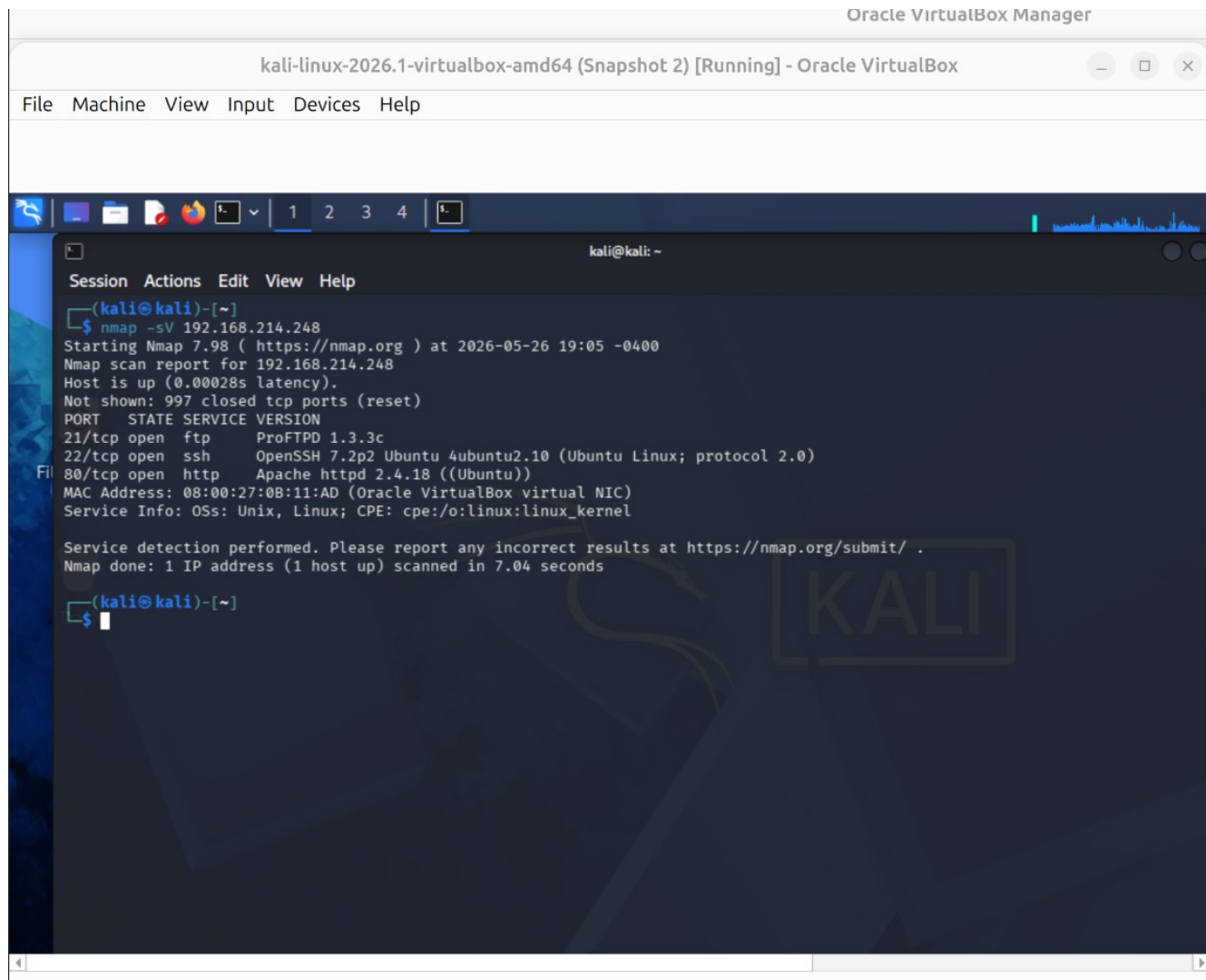


```
csec (Snapshot 3_DVWA_clean_install) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Terminal
marlinspike@vtcsec: ~
marlinspike@vtcsec:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:0b:11:ad brd ff:ff:ff:ff:ff:ff
    inet 192.168.11.248/24 brd 192.168.11.255 scope global dynamic enp0s3
        valid_lft 3569sec preferred_lft 3569sec
    inet6 fe80::20b2:f3aa:e452:869e/64 scope link
        valid_lft forever preferred_lft forever
marlinspike@vtcsec:~$
```

*\*P.S. this screenshot is strictly illustrative and does not contain the actual IP addresses used for the lab.*

## 2. nmap Scans

For enumeration, the target machine was scanned with commands 'nmap -sV <ip address>', and 'nmap -A <ip address>'



The screenshot shows the Oracle VM VirtualBox Manager interface. The main window displays a terminal window titled 'kali@kali: ~'. The terminal output shows the execution of the command 'nmap -sV 192.168.214.248'. The output includes the Nmap version (7.98), the target IP address (192.168.214.248), and the scan results. The results show three open ports: 21/tcp (ftp, ProFTPD 1.3.3c), 22/tcp (ssh, OpenSSH 7.2p2 Ubuntu 4ubuntu2.10), and 80/tcp (http, Apache httpd 2.4.18 ((Ubuntu))). The terminal also displays the MAC address (08:00:27:0B:11:AD) and the OS service information (OS: Unix, Linux; CPE: cpe:/o:linux:linux\_kernel). The terminal window is titled 'kali@kali: ~' and has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'.

```
kali@kali: ~  
$ nmap -sV 192.168.214.248  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-26 19:05 -0400  
Nmap scan report for 192.168.214.248  
Host is up (0.00028s latency).  
Not shown: 997 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
21/tcp    open  ftp      ProFTPD 1.3.3c  
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.10 (Ubuntu Linux; protocol 2.0)  
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))  
MAC Address: 08:00:27:0B:11:AD (Oracle VirtualBox virtual NIC)  
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 7.04 seconds  
  
kali@kali: ~  
$
```

The **nmap -sV** scan is a basic version scan that returns the services and their versions being run on the machine. As seen above, the ports are named, their states and the versions. You can also see the MAC address for the VirtualBox tool as well as OS service information.

```
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ . 168.146.248/
Nmap done: 1 IP address (1 host up) scanned in 7.10 seconds

(kali@kali)-[~]
$ nmap -A 192.168.146.248
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-22 06:35 -0400
Stats: 0:00:07 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 66.67% done; ETC: 06:35 (0:00:03 remaining)
Nmap scan report for 192.168.146.248
Host is up (0.00025s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.3c
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.10 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
| 2048 d6:01:90:39:2d:8f:46:fb:03:86:73:b3:3c:54:7e:54 (RSA) /MA35
| 256 f1:f3:c0:dd:ba:a4:85:f7:13:9a:da:3a:bb:4d:93:04 (ECDSA)
| 256 12:e2:98:d2:a3:e7:36:4f:be:6b:ce:36:6b:7e:0d:9e (ED25519)
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
|_ http-title: Login :: Damn Vulnerable Web Application (DVWA)
|_ Requested resource was login.php
|_ http-cookie-flags:
|_ /:
|_ PHPSESSID:
|_ httponly flag not set
|_ http-robots.txt: 1 disallowed entry
|_ /
|_ http-server-header: Apache/2.4.18 (Ubuntu)
|_ http-git:
|_ 192.168.146.248/.git/
|_ Git repository found!
|_ Repository description: Unnamed repository; edit this file 'description' to name the ...
|_ Remotes:
|_ https://github.com/digininja/DVWA.git
|_ Project type: PHP application (guessed from .gitignore)
MAC Address: 08:00:27:0B:11:AD (Oracle VirtualBox virtual NIC)
Device type: general purpose|router
Running: Linux 4.x|5.x, Mikrotik RouterOS 7.x
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), Mikrotik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel curity Level: low

TRACEROUTE
HOP RTT ADDRESS
1 0.25 ms 192.168.146.248

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.78 seconds
```

The **nmap -A** scan above is referred to as the aggressive scan because it outputs a more detailed intel on the target. In addition to an -sV scan, the -A scan reveals the OS fingerprint/CPE, runs NSE scripts and maps the physical path to the target VM using Traceroute. You find all this information revealed in the command output above.

## PHASE 2- Web Enumeration

### 2a. Gobuster

In the first part of this phase, **gobuster** was used to carry out web enumeration. Gobuster is a tool used to brute-force websites to gain access to hidden files, directories and subdomains. Here's the result of using gobuster on the target machine:

```
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help
└─$ gobuster dir -u http://192.168.146.248 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.146.248
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 280]
.git/HEAD (Status: 200) [Size: 23]
.htpasswd (Status: 403) [Size: 280]
.htaccess (Status: 403) [Size: 280]
config (Status: 301) [Size: 319] [→ http://192.168.146.248/config/]
database (Status: 301) [Size: 321] [→ http://192.168.146.248/database/]
docs (Status: 301) [Size: 317] [→ http://192.168.146.248/docs/]
external (Status: 301) [Size: 321] [→ http://192.168.146.248/external/]
favicon.ico (Status: 200) [Size: 1406]
index.php (Status: 302) [Size: 134] [→ login.php]
php.ini (Status: 200) [Size: 154]
phpinfo.php (Status: 302) [Size: 134] [→ login.php]
robots.txt (Status: 200) [Size: 25]
server-status (Status: 403) [Size: 280]
tests (Status: 301) [Size: 318] [→ http://192.168.146.248/tests/]
Progress: 4613 / 4613 (100.00%)

Finished

(kali@kali)-[~]
$
```

First, we see that gobuster opened in directory enumeration mode and proceeded to scan the target '192.168.146.248' using the 'common.txt' wordlist fed in the command-line. The rest of the output includes:

**Status 200 files-** These are files that exist and are fully accessible

- **.git/HEAD:** This indicates that the git repository is exposed and vulnerable to attack. It can be dumped, altered or source codes altered. Also, assets like database elements, API keys or hidden endpoints are potentially accessible.
- **php.ini:** The PHP configuration file is exposed. This leaks server configuration details and PHP environment settings.

- **robots.txt**: This is a standard file used to tell search engines what not to crawl. Attackers check this first to find hidden paths.

### Status 301/302 directories - These paths redirect to other locations

- **config / database**: these are high-priority directories. They would likely contain configuration files or database connection scripts that would be interesting to attackers.
- **index.php / phpinfo.php**: Both redirect to `login.php`. This indicates an authentication portal is active. `phpinfo.php` usually leaks intense server details if it can be bypassed.
- **docs / external / tests**: General directories that might hold documentation, test scripts, or third-party code with known vulnerabilities.

**Status 403 (Forbidden / Access Denied)**- These files exist on the server but the current request lacks permission to view them.

- **.hta / .htaccess / .htpasswd**: Standard Apache configuration and password files. These are blocked by default.
- **server-status**: This is an Apache server status page that is usually restricted to localhost.

## PHASE 3- VULNERABILITY TESTING

### 3a. OS Command Injection

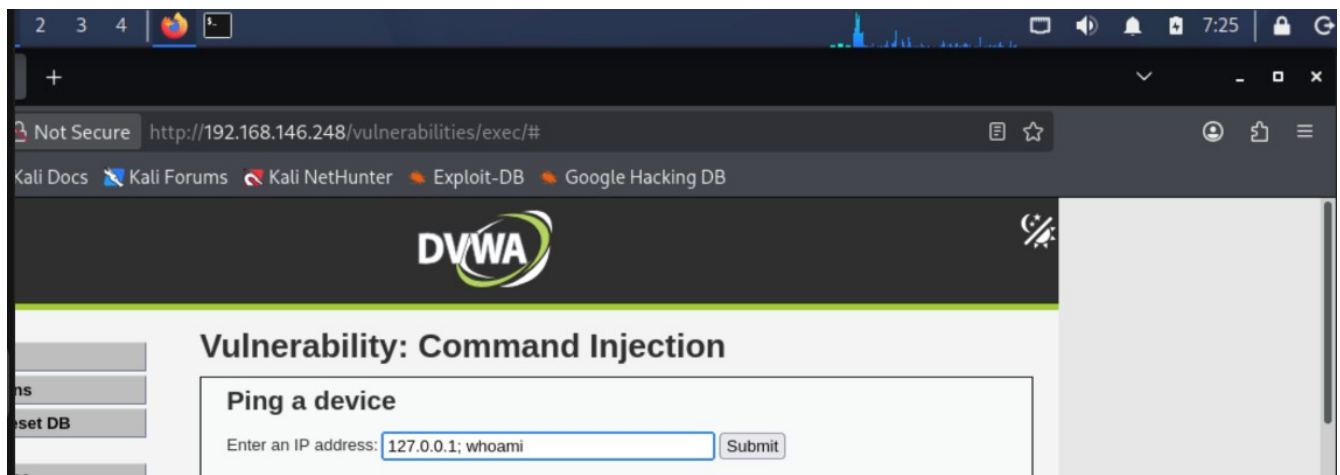
i.

**Severity**: High

**Description**: The application passed a user-controlled input into a system shell command without sanitization.

**Affected Endpoint**: `/vulnerabilities/exec/`

**Payload applied**: `127.0.0.1; whoami`, an active command injection. It is called 'active' because it gives an immediate and visible response.

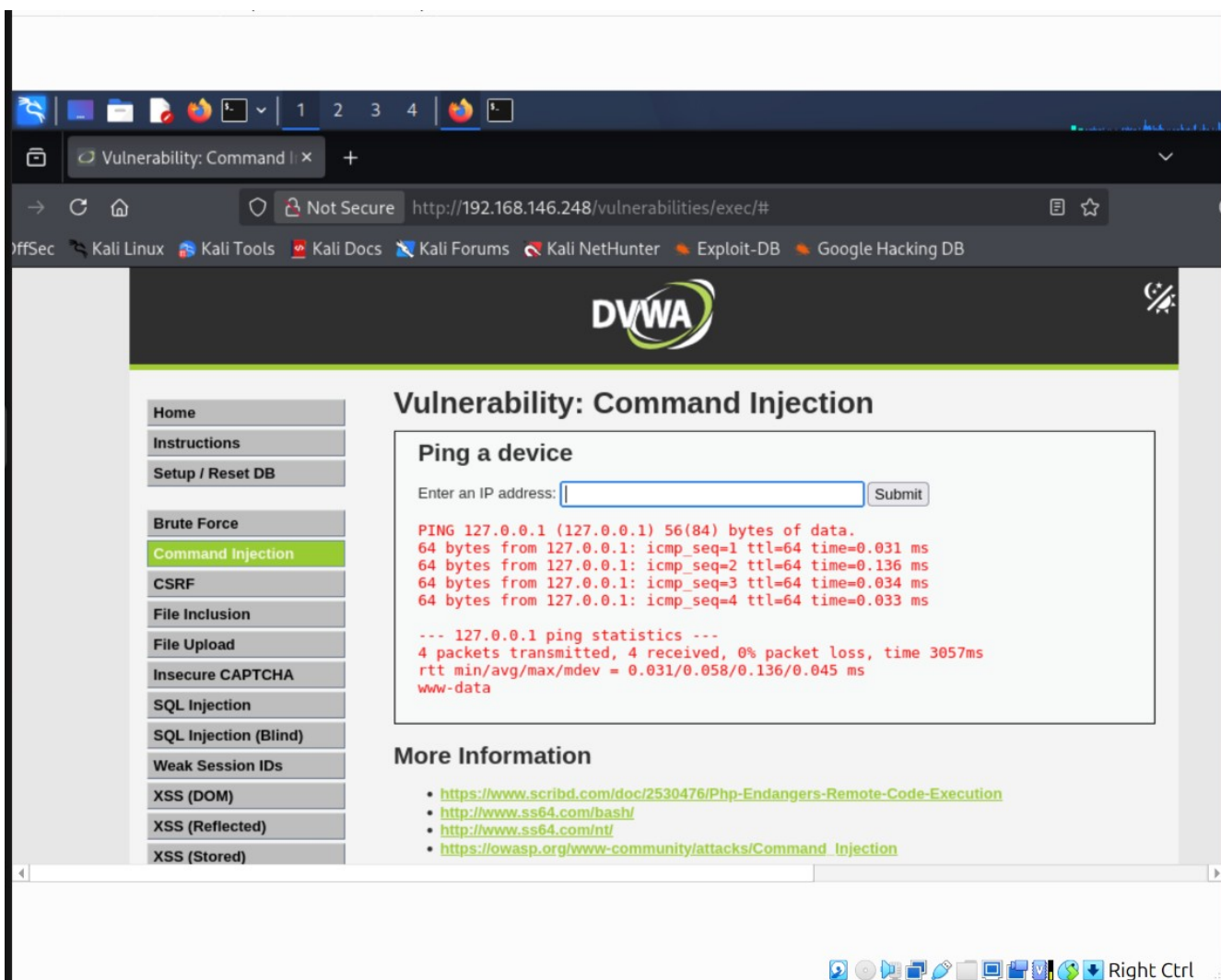


### Output:

The OS command ran successfully and the following were observed:

- the local ip address 127.0.0.1 was pinged four times successfully, with zero packet loss.
- the injected payload ran as **www-data**, meaning that the web server was isolated from the rest of the system, preventing attackers from gaining full root control over the whole server. Though the command injection was successful, compromise remains limited to the web server's permissions.





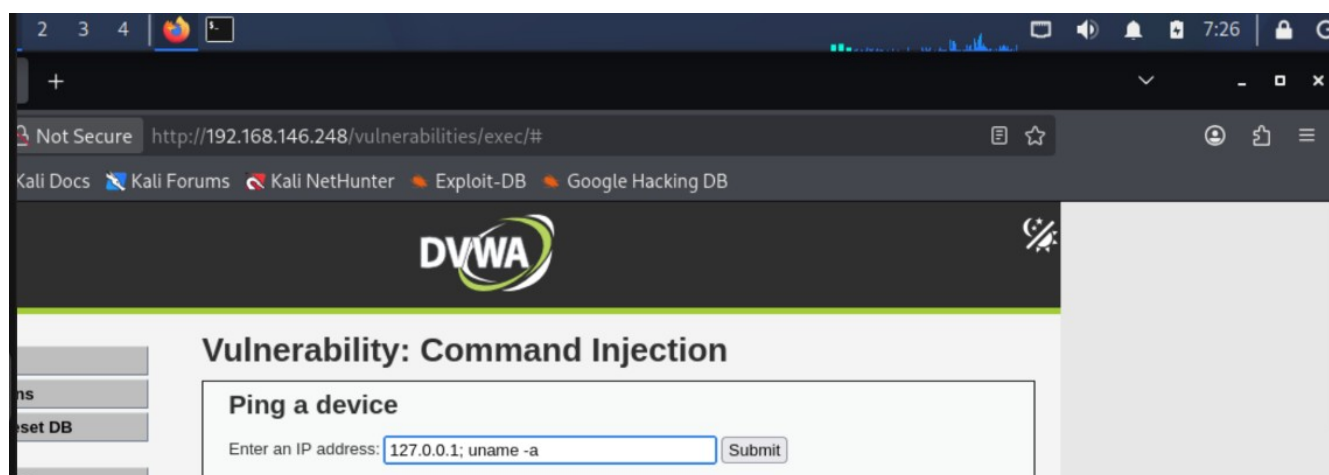
ii.

Severity: High

Description: A recon command was used to gather information about the server's OS kernel.

Affected endpoint: same

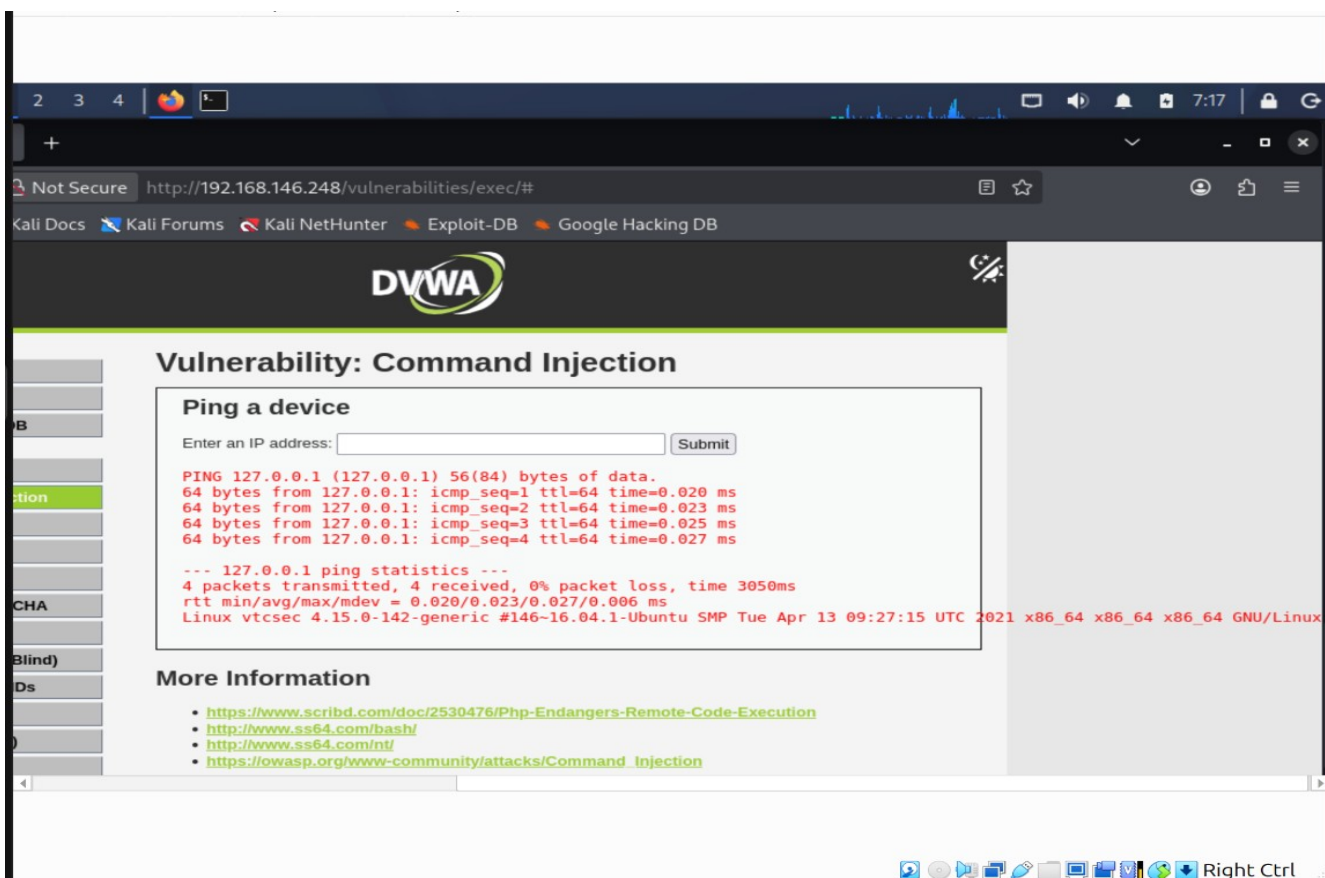
Payload applied: 127.0.0.1; uname -a, also an active command.



### Output:

As before, there were 4 successful pings with zero packet loss. The injected payload however returned detailed system reconnaissance as follows:

- Linux: The operating system name.
- vtcsec: The hostname of the target machine.
- 4.15.0-142-generic: The specific Linux kernel version currently running.
- #146~16.04.1-Ubuntu: The version compilation details, revealing the host is running Ubuntu 16.04.
- UTC 2021: The build timestamp of the kernel.
- x86\_64: The machine's hardware architecture (64-bit).



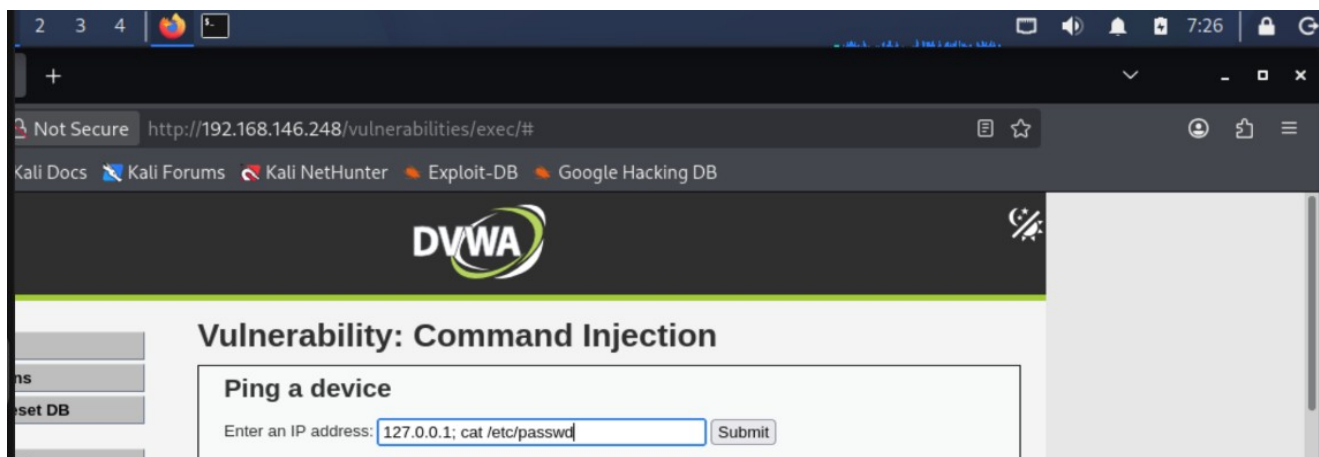
iii.

**Severity:** Critical

**Description:** Another command injection was used to ping the local loopback address and a payload that exposed critical system user information.

**Affected endpoint:** same as before

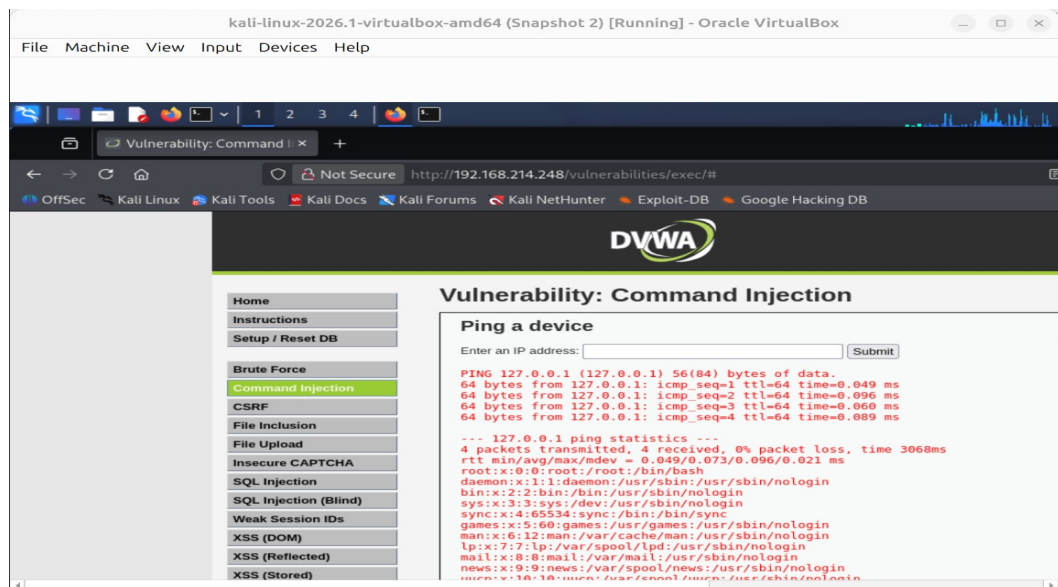
**Payload applied:** 127.0.0.1; cat /etc/passwd, also a direct/active command

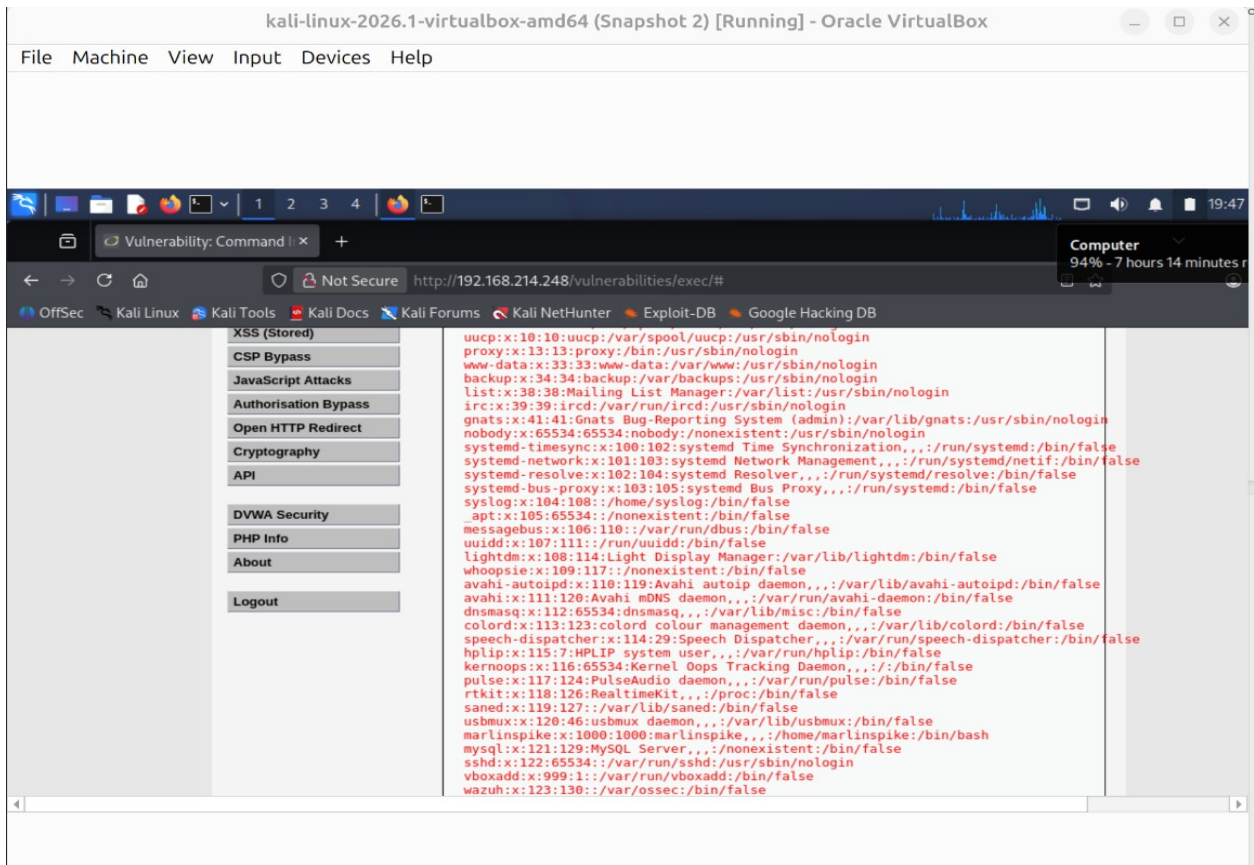


Output:

Critical system user details were exposed such as:

- **root:x:0:0:root:/root:/bin/bash** (The root administrator account)
- **daemon, bin, sys, and sync** system service accounts.
- **www-data (UID 33)**: This is the service account running the Apache/Nginx web server and DVWA. Any command injection payloads you execute run with this user's limited permissions, not root.
- **marlinspike (UID 1000)**: This is a standard human user account on the machine. Notice it has a home directory (/home/marlinspike) and a valid interactive login shell (/bin/bash). This would be a primary target for privilege escalation.
- **Service Accounts**: The system is running several notable services, including **MySQL/MariaDB** (database), **sshd** (Secure Shell for remote access), and **wazuh** (a security monitoring and endpoint detection agent). See images below.





## Implications:

- The application is vulnerable to RCE: Remote Code Execution (server compromise/sensitive data exposure and/or exfiltration)
- Persistence & privilege escalation: the damage is still contained, attackers cannot immediately delete system files, install malware at root level or access other users' data. However, www-data is often used as a stepping stone for attackers.
- They looked for database passwords and can attempt local privilege escalation on several targets like marlinspike, to become root.
- An attacker can swap uname -a for a different payload that gives them a reverse shell; an interactive terminal window into the environment.
- Internal network scanning, lateral movement.
- for uname -a specifically, kernel versions information has been gathered to aid in LPE exploits

### 3b. SQL Injection

We tried 2 different SQL queries to trick the application into revealing secret information, specifically ID logins and hashes. Their output is shared below.

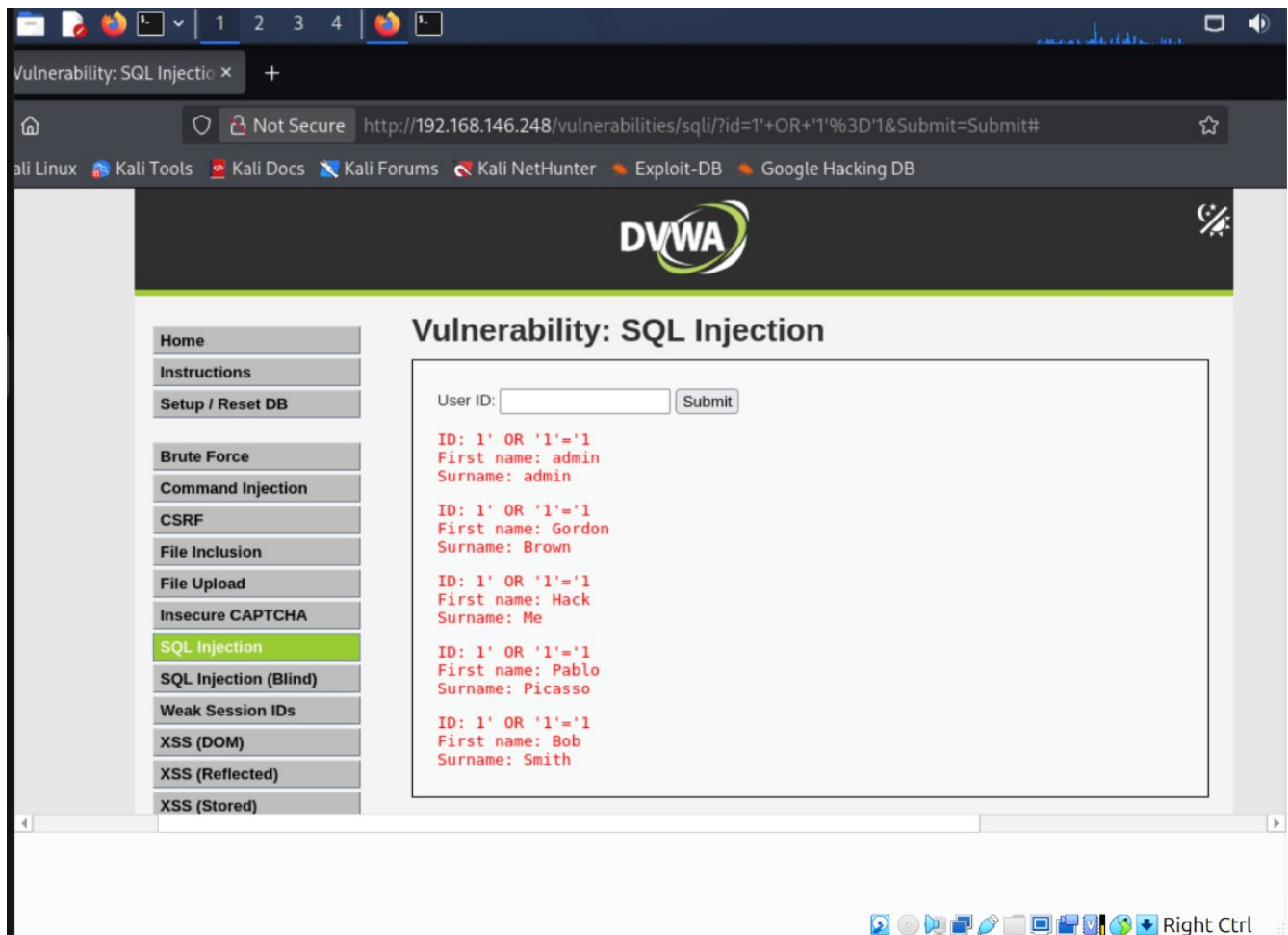
i.

**Severity:** Critical

**Description:** This vulnerability allows an attacker bypass login controls without valid credentials, allowing retrieve records without authentication.

**Affected endpoint:** <http://192.168.146.248/vulnerabilities/sqli> via id GET

**Payload applied:** " 1' OR '1'='1 "



**Implications:**

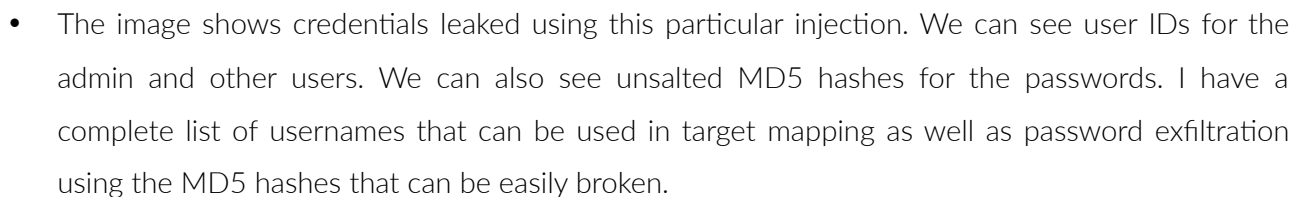
- The image above shows a successful SQL injection and the attack has retrieved records containing user IDs. Full names, user names and account structures have been exposed.
- These users can become targets of brute-force attacks or spear phishing campaigns.



- ii.

**Description:** This vulnerability allows an attacker extract sensitive data from a database, in this case displaying user credentials directly on screen.

Payload applied: " 1' UNION SELECT user,password FROM users#"



- I can carry out a privilege escalation using the admin account hash by pivoting to a systems administrator.
- Again, RCE is another course of action as well as
- Lateral movement and session hijacking especially if users have repeated passwords across services.

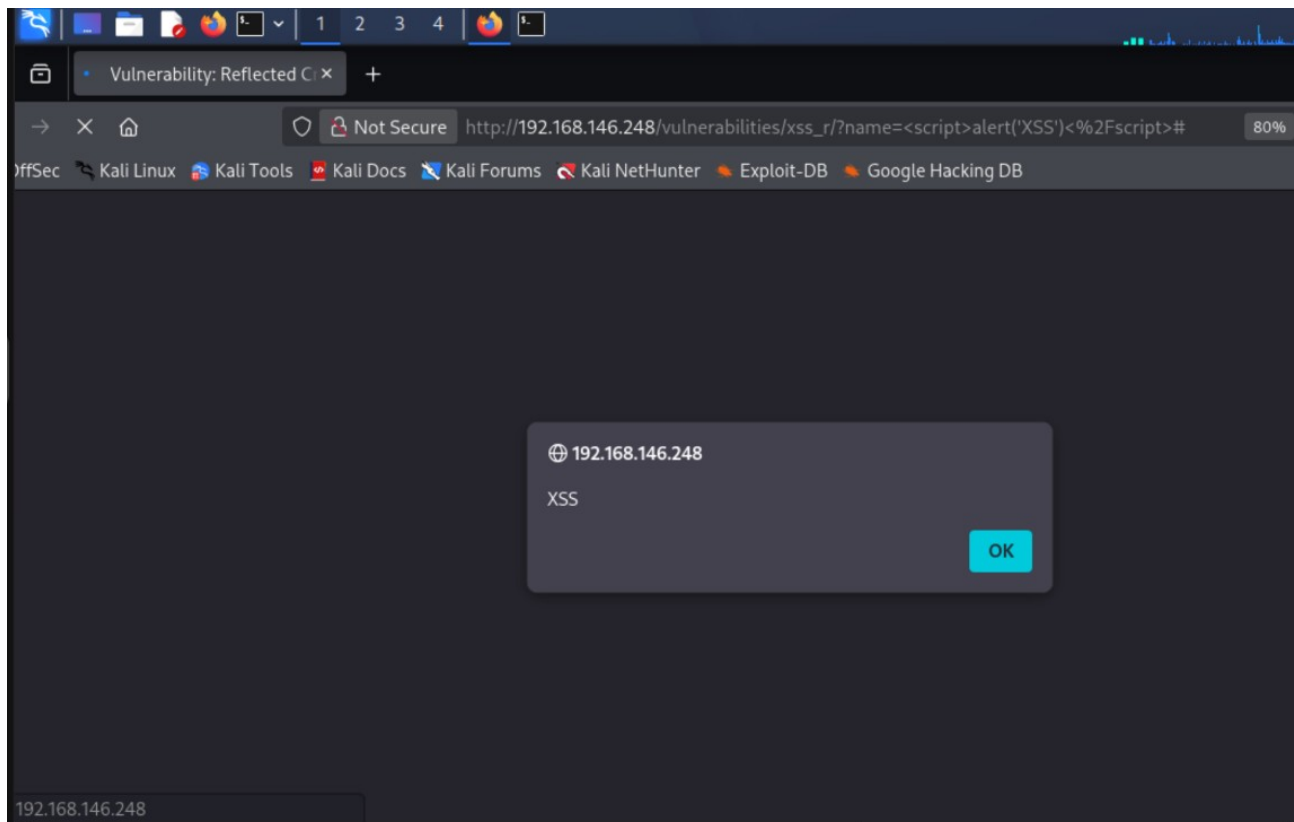
### 3c. Cross-Site Scripting (XSS) Reflected

Severity: Medium

**Description:** This is an attack in which an attacker injects malicious script into an otherwise trusted website. It is a browser level attack and XSS Reflected in particular is a non-persistent scripting type where the payload is never stored on the server but is visible to the user as a link.

**Affected endpoint:** [http://192.168.146.248/vulnerabilities/xss\\_r/..](http://192.168.146.248/vulnerabilities/xss_r/)

**Payload applied:** “ `<script>alert('XSS')</script>` ”





## Implications:

- A successful XSS injection can lead to session hijacking by using active session cookies to access a user's account without a password.
- Sensitive information can be harvested while the user is browsing.
- It can be used for phishing or redirecting users to malicious sites.

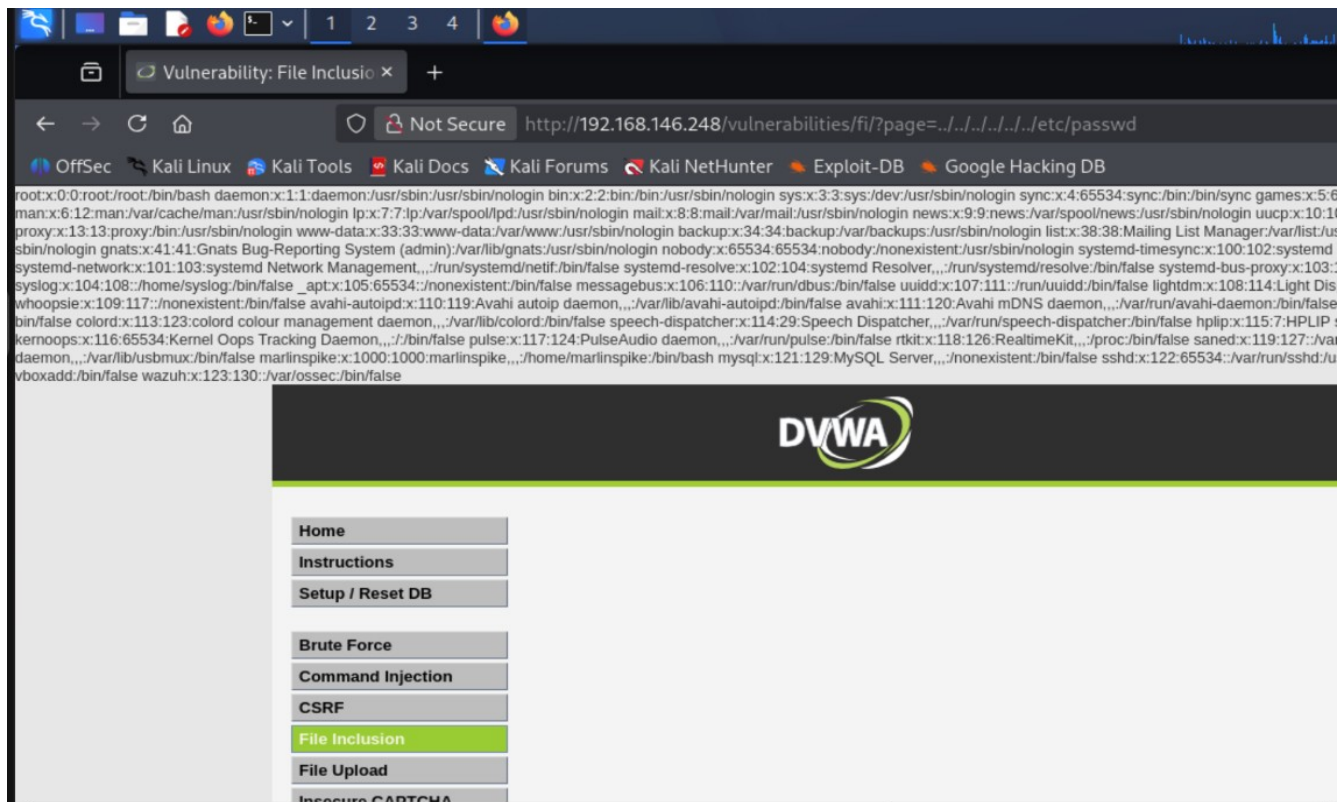
## 3d. File Inclusion

Severity: High

**Description:** File inclusion allows an attacker read sensitive server files or execute malicious code. In this example, the attacker has accessed user accounts registered on the server.

**Affected endpoint:** [http://192.168.146.248/vulnerabilities/fi/..](http://192.168.146.248/vulnerabilities/fi/)

**Payload applied:** “ ../../../../../../etc/passwd”



## Implications:

The image shows a successful local file inclusion with directory traversal up to 6 folders deep. I was able to access registered user accounts stored in the system root. The block of text at the top in the injection is the `etc/passwd` file on display.

## REMEDIATION:

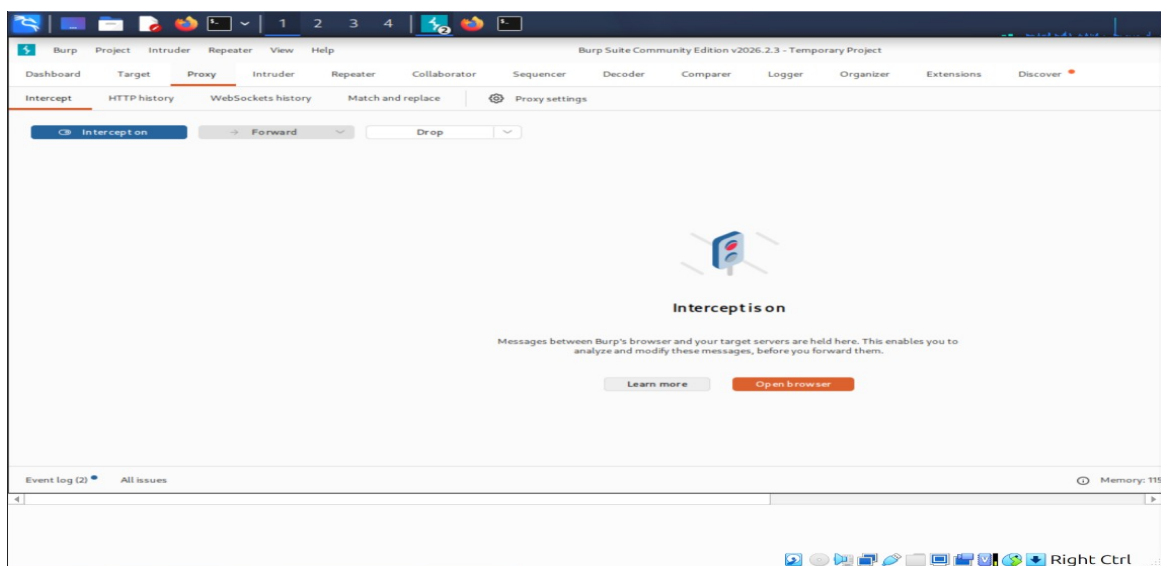
For these vulnerabilities, the following will apply to secure the application:

- Avoid shell execution. Use inbuilt libraries of the programming language instead.
- Create an allow list aka whitelisting or strict input validation.
- Sanitize user input i.e actively alter input to remove dangerous components or escaping by adding special characters that will treat the input as harmless text.
- For SQL injections in particular, a good remediation will be to use prepared statements so that the database uses user input as a literal value only. This is done by using placeholders for user input values instead of adding them into query text directly, separating query logic from the data itself. Other remediation steps include user input validation, web app firewalls that can block common attack patterns like ' OR 1=1', or escaping user input using certain special characters eg '/
- Implement least privilege permissions.
- To specifically remediate against file inclusion requires restricting how files are dynamically referenced. The following steps are recommended:
  - **Use an Allowlist:** Avoid direct user input for filenames. Maintain a strict list of permitted files (e.g., home, about, contact) and reject any input that does not exactly match.
  - **Disable Remote Execution:** do not allow the runtime environment load external files. In PHP, for instance, ensure that 'allow\_url\_include' is explicitly set to Off in the system configuration.
  - **Path Sanitization:** Use built-in programming functions to strip off directory traversal characters like ../ and isolate the script execution to a dedicated directory.

## PHASE 4: HTTP TRAFFIC ANALYSIS AND INTERCEPTION WITH BURP SUITE

The process involved installing Burp Suite between the browser and the target application, i.e. between Firefox and DVWA. The tool was then used to intercept traffic as well as replay some of the traffic flows. The steps taken were;

- a) Launch burpsuite in CLI
- b) Ensure that the Listener in Burp was running by checking for 127.0.0.1:8080 under proxy options.
- c) Configure Proxy in Firefox settings by setting configuration to manual proxy configuration and entering the parameters as found in Burp, i.e. 127.0.0.1 and port to 8080 and then checking 'use this proxy for all servers'.
- d) Save settings (N.B.: Be careful undo these changes when done with this exercise else it affects the functionality of the browser.)
- e) Go back to burp and turn on 'Intercept'



4b.

Turning intercept on made DVWA nonresponsive. The image below shows the details of the intercepted requests made to DVWA in Firefox browser.

